

Technology Stack Document

Project Technology Stack — Overview & Rationale

1. Final Technology Stack

Layer	Selected Technology
Frontend Framework	Next.js (App Router) + TypeScript
UI Framework	Tailwind CSS + shadcn/ui
State Management	TanStack Query + Redux Toolkit
Data Tables	TanStack Table
Drag & Drop	dnd-kit
Forms	React Hook Form + Zod
Charts	Recharts
Backend	Python + FastAPI
Database	PostgreSQL + pgvector
ORM	SQLAlchemy + Alembic
Authentication	JWT (python-jose) + Passlib (Argon2)
File Storage	AWS S3
Cache	AWS ElastiCache
Background Jobs	AWS SQS
Frontend Hosting	Vercel
Backend Hosting	AWS ECS / Fargate
Database Hosting	Amazon RDS (PostgreSQL)
CI/CD	GitHub Actions

Alternatives are covered in Section 2 only for the core architecture decisions — Frontend Framework, Backend, and Database — since these choices have the largest long-term impact on the project. Other layers list the selected technology only.

2. Technology Details, Rationale, Pros & Cons

Each layer below shows the selected technology with its pros and cons. For the core architecture layers noted above, an alternative that was considered is also included.

1. Frontend Framework

Selected: Next.js (App Router) + TypeScript

Chosen for its scalable file-based routing, built-in API routes, and strong deployment support on Vercel — giving the project better long-term structure than a plain single-page app.

Pros	Cons
• Built-in routing and file-based structure • Strong ecosystem and developer experience • Better long-term project organization	• Can feel like overkill for simple apps that don't need SEO

Alternative Considered: React (Vite)

A lighter, simpler option considered for its speed, but it needs more manual setup as the project grows.

Pros	Cons
• Very fast local development • No server-rendering complexity • Simple for small apps	• Needs manual routing setup • No built-in API routes • More upfront setup decisions

2. UI Framework

Selected: Tailwind CSS + shadcn/ui

Chosen for fast, consistent styling with accessible, customizable components — good for building a polished UI quickly.

Pros	Cons
• Fast styling and iteration • Highly customizable design system • Accessible components out of the box	• Verbose class names in markup • Slight learning curve for new developers

3. State Management

Selected: TanStack Query + Redux Toolkit

TanStack Query handles server data and caching efficiently, while Redux Toolkit manages complex client-side state predictably.

Pros	Cons
• Strong caching and data syncing • Predictable, centralized state	• Redux adds some boilerplate • Can be overkill for simple state needs

4. Data Tables

Selected: TanStack Table

A headless table library chosen for flexibility and performance, giving full control over the UI.

Pros	Cons
• Highly flexible, full UI control • Handles large datasets well	• Requires manual UI implementation • Steeper learning curve

5. Drag & Drop

Selected: dnd-kit

Chosen for being lightweight, accessible, and easy to customize.

Pros	Cons
• Lightweight footprint • Highly accessible • Fully customizable	• Less plug-and-play than older libraries like react-beautiful-dnd

6. Forms

Selected: React Hook Form + Zod

Chosen for strong runtime performance and type-safe schema validation, keeping form data reliable across the app.

Pros	Cons
• Fast, minimal re-renders • Strong type safety with schema validation • Easy validation setup	• Typing can get complex for reusable form components

7. Charts

Selected: Recharts

Chosen for being composable and built for React, making it easy to match the product's design system.

Pros	Cons
• Composable, React-native components • Easy to customize	• Can struggle with very large datasets (SVG rendering)

8. Backend

Selected: Python + FastAPI

Driven by planned AI integrations, FastAPI was chosen for its mature AI/ML ecosystem, speed, and built-in validation with async support.

Pros	Cons
• Mature, native AI/ML ecosystem • Very fast performance	• Smaller ecosystem for traditional enterprise tooling than Node/Java

• Built-in request validation (Pydantic)	
--	--

Alternative Considered: Node.js + NestJS

Considered to keep the whole stack in one language with strong enterprise architecture, but has a weaker AI/ML ecosystem.

Pros	Cons
• Strong enterprise-grade architecture • Keeps entire stack in one language (TypeScript)	• Weaker AI/ML support • More verbose, decorator-based syntax

9. Database

Selected: PostgreSQL + pgvector

Chosen for strict relational data handling, referential integrity, and complex joins — while also supporting vector search in the same database.

Pros	Cons
• Strong relational model with referential integrity • Handles complex joins reliably • Native vector search	• Vertical scaling limits • Vector search is slower than dedicated vector databases at large scale

Alternative Considered: MongoDB

Considered for its flexible schema and horizontal scaling, but a poor fit for this project's relational data.

Pros	Cons
• Flexible schema • Fast document retrieval • Easy horizontal scaling	• Poor fit for highly relational data • No strict foreign keys • Complex multi-document transactions

10. ORM

Selected: SQLAlchemy + Alembic

Chosen for fine-grained SQL control and mature migration tooling via Alembic.

Pros	Cons
• Powerful and mature • Fine-grained SQL control	• Steep learning curve • Can feel heavy for simple queries

11. Authentication

Selected: JWT (python-jose) + Passlib (Argon2)

Chosen for a stateless, scalable approach that's quick to implement, with Argon2 hashing for strong password security.

Pros	Cons

• Stateless and scalable • Fast to implement	• Harder to instantly revoke tokens without a blocklist
---	---

12. File Storage

Selected: AWS S3

The industry-standard object storage solution, offering high durability at low cost.

Pros	Cons
• Industry standard and highly durable • Cost-effective at scale	• AWS IAM and permissions can be complex to configure

13. Cache

Selected: AWS ElastiCache

A fully managed caching service with instant high availability and automated backups, removing manual patching and monitoring work.

Pros	Cons
• Extremely fast performance • Fully managed and scalable	• Adds infrastructure cost and architectural complexity

14. Background Jobs

Selected: AWS SQS

A serverless, highly durable, and scalable managed queue that decouples jobs without relying on a tightly coupled Redis instance.

Pros	Cons
• Fully managed and highly durable • High throughput • Decouples services cleanly	• At-least-once delivery requires idempotent workers

15. Frontend Hosting

Selected: Vercel

Chosen for zero-config deployments and excellent CI/CD integration, tightly matched to the Next.js framework.

Pros	Cons
• Zero-config deployments • Excellent CI/CD integration	• Vendor lock-in • Can get expensive at scale

16. Backend Hosting

Selected: AWS ECS / Fargate

Selected for serverless container management that is highly scalable and reliable.

Pros	Cons
• Serverless containers • Highly scalable and reliable	• Complex AWS networking/IAM setup

17. Database Hosting

Selected: Amazon RDS (PostgreSQL)

Chosen for automated backups and managed scaling, reducing operational overhead for the team.

Pros	Cons
• Automated backups • Managed scaling	• Higher cost compared to self-hosting or a simple VPS

18. CI/CD

Selected: GitHub Actions

Selected for its tight integration with the code repository and high customizability of pipelines.

Pros	Cons
• Integrated directly with the repository • Highly customizable	• Difficult to debug complex pipelines locally